Junior Java Backend fejlesztő III.

OOP alapok

Botlik Béla

Osztályok, objektumok, objektum-orientáció

(A prezentációban felhasználásra kerültek diák a Ruander Oktatóközpont korábbi tananyagából. Szerzők: Béres Gábor, Czigléczky Gábor, Fehér Róbert, Kis Balázs)

Bevezetés I.

- Az OO programozás egy merőben más szemlélet, mint az összes előző technika:
  - Megvalósítja a személyiség független programozás lehetőségét.
  - Elősegíti a csapatmunka létrejöttét.
  - Lehetővé teszi a pontos tervezést:
    - A tervezés során pontosan megszabható, hogy a később létrejövő program pontosan mit fog tudni, és melyik részegység pontosan hogyan fog részt venni a futásban.
    - Az OOP segítségével lehetőség nyílik a több absztrakciósszintű tervezésre is.
- Elősegíti a könnyen tovább fejleszthető és javítható programok elkészítését.

Bevezetés II.

- Az Objektum-Orientált programozásban alapvetően a következő elemek, fogalmak jelennek meg:
  - Osztályok
  - Osztályok hierarchiája
  - Osztályok csoportosítása
  - Objektumok
- Természetesen ezek csak az alapvető elemek, későbbiekben még sok más elemmel fog bővülni a teljes OO elem- és fogalomtár.

Osztály I.

- Az osztály olyan programrész, amelyben az alprogramok (tagmetódusok) és a változók (adattagok) egyetlen közös feladat érdekében működnek.
- Az osztályban lévő egyes elemek, egymás nélkül nem tudnak működni, vagy nincs értelme működtetni őket.
- Például, ha létrehozunk egy osztályt mely fájlok feldolgozását teszi lehetővé, az osztályban szereplő változók (adatszerkezetek) fogják tárolni a feldolgozandó és feldolgozott adatokat, míg az alprogramok valósítják meg a feldolgozást. De önmagában csak a tárolás, vagy csak a feldolgozás nem működőképes.

Osztály II.

- Így tehát az osztály, mint egy tervrajz leírja, hogy a belőle létrejövő objektumok mit fognak tudni, mit fognak tudni tárolni, milyen alprogramjaik, tulajdonságaik lesznek.
- Programozás közelibb szinonimával jelezve, az osztály lesz maga a típus (referencia típus) OOP környezetben.
  - Megszabja, hogy a belőle létrejövő objektum mire képes, mit tud tárolni, milyen tulajdonságai lesznek.

Objektum/példány

- Az objektum, vagy más néven példány, egy egyedi leképezése az osztálynak.
- Azért egy leképezése, mert az alprogramok, a változók és egyéb elemek ugyanazok mint az osztályban, azonban a példány mégis egyedi, mivel az adott változók értéke, és ily módon az egyes alprogramok futása, vagy visszaadott értéke különböző lehet két vagy több objektum esetében. Tehát minden példány önálló életet él.

Osztály és objektum kapcsolata

- Valójában a kettő kapcsolatát úgy lehetne leírni, hogy az osztály egy tervrajz, mely megszabja, hogy a belőle létrejövő példány mit kell tudjon, milyen tulajdonságai vannak.
- Azonban azt, hogy az adott tulajdonság milyen értéket képvisel az adott objektumokban, már teljesen egyedi.
- Az objektum az osztály példányosítása útján jön létre:
  - `<Osztály> <azonosító> = new <Osztály>(<paraméterek>);`

Package I.

- A package, vagy csomag valójában egy osztály csoportosító eszköz.
- Feladata, hogy a programban szereplő osztályokat, feladatkörük, illetve felhasználási jellegük szerint csoportokba ossza, mely elősegíti a programban való használatot, hiszen tudjuk hol keressük, illetve a későbbi újrafelhasználás céljából is jó, hiszen az összetartozó osztályokat együtt lehet hordozni.

Package II.

- További fontos hatása, hogy a csomagok segítségével szabályozható, hogy az adott programhoz, osztályhoz, milyen részegységek (package-k) fognak kelleni, és ennek megfelelően, miket kell betölteni memóriába.
- Ez a megoldás már a legtöbb OOP alapú programozási nyelvben megtalálható, azonban nem feltétlenül ugyanígy hívják.
  - Például C#-ban namespace (névtér) a neve.

OOP I.

- Természetesen az egyes elemek felhasználásával történik az OO programozás maga, de önmagában az elemek felhasználása nem garantálja, hogy a programunk megfelelően használja fel az OO programozás nyújtotta előnyöket.
- Az Objektum-Orientált Programozásnak 6 fő kritérium eleme létezik, melyek felhasználása eredményezi az OO nyújtotta előnyöket.

OOP II.

- Kritérium elemek:
  - Absztrakció
  - Egységbezárás
  - Adatrejtés
  - Öröklés
  - Polimorfizmus
  - Kód újrafelhasználhatóság

Absztrakció

- Absztrakció, mint kritérium elem már a program tervezési szakaszában alkalmazandó.
- Az absztrakció gyakorlatilag egy lényegkiemelés, melynek a tervezés szempontjából fontos szerepe van.
- Lényege, hogy megtervezzük programunk milyen osztályokból fog felépülni, figyelembe véve az esetleges későbbi továbbfejlesztési lehetőségeket is.
- Ez az eddigiekkel szemben egy új szemlélet, hiszen eddig célfeladat alapján terveztük és írtuk meg programjainkat.

Egységbezárás

- Valójában azt jelenti, hogy az objektumhoz tartozó adatokat és alprogramokat (funkcionalitást) egybegyűjtjük az osztályba.
- Ezáltal elérhető, hogy az adott osztályba implementáljuk az összes olyan elemet, amik egymás nélkül nem képesek működni.
- Másrészről biztosítható az újrahasználhatóság is, hiszen az osztály biztosítja, hogy az objektum saját maga legyen felelős az összes benne tárolt adatért is.
  - Ezzel együtt jár, hogy az osztályba implementálni kell az összes olyan alprogramot, ami a benne lévő adatok módosításához kell.

Adatrejtés

- Az osztályon belül elrejtjük azokat az alprogramokat és adatokat, amiket a külvilág (más objektumok) számára mi nem szeretnénk publikusan kezelni, illetve publikussá tesszük azokat, amik kívülről is elérhetők.
- Erre azért van szükség, mert így kontrollálható, hogy egy objektum melyik másik objektum adataihoz férhet hozzá.
- Főként csapatmunka során, illetve a továbbfejleszthetőség érdekében fontos.

Öröklés

- Egy specializációs művelet, melynek eredményeképpen, az egymásból következő osztályokat, egymásból származtatjuk, így megspórolva a külön fejlesztési időt.
- A specializálódás is jól mutatja, hogy az öröklődés mindig az általánosabb osztály felől, mint ős felől halad a specializáltabb leszármazottak felé.
- Pl.: Van egy már megírt Jarmu osztály, és szeretnénk megírni az Auto és a Repulo osztályt, akkor mindkét osztályt a Jarmu-ből fogjuk származtatni.

Polimorfizmus

- Többalakúság melynek alapvetően két jelentése is van:
  - Lehetőséget biztosít, hogy minden osztályból létrehozott tároló képes legyen tárolni olyan objektumokat, melyek valamelyik, az adott osztály leszármazott osztályából jöttek létre.
    - Pl.: Jarmu val = new Auto(…); Jarmu val = new Repulo(…);
      - Ez azért lehetséges, mert minden osztály leszármazottja egy specializált ős típusú elem, így tehát az előző esetben minden autó valójában egy speciális jármű.
  - Mindezek mellett a polimorfizmus megengedi, hogy az egyes leszármazottak - habár együtt kezelhetők az ős típus szerint- mégis bizonyos feladatokat más-más módon valósítsanak meg.
    - Pl.: A Jarmu osztály megszabja, hogy minden járműnek legyen Inditas() funkciója, de egy Repulo és egy Auto is másképp indul, így maga az Inditas() mindegyikben szerepel, de mindkettő másképpen valósítja azt meg.

Kód újrafelhasználhatóság

- Az objektum orientált programok későbbi felhasználása sokszorta egyszerűbb, mint a többi technológia alapján előállított programoké, hiszen az osztályok függetlenségéből adódóan könnyedén átvehetők programrészek -vagy akár komplett kódok is- más programokba.

Valósvilág modell I.

- Az objektumorientált programok fejlesztésénél elsődleges cél, hogy a modell felépítése során, meglévő objektumokból kell OO programozásban használatos objektumokat létrehozni.
- Ezt a valósvilág modell segítségével tudjuk elérni, ahol a meglévő objektumok a valósvilág objektumai (pl.: autó, ember, labda, számítógép, stb.), a célkörnyezetben pedig ezeknek kell megjelennie (OOP szerint).

Valósvilág modell II.

- A valóság modellezéséhez egy szempontrendszer kell, melynek segítségével tudjuk a VV objektumot modellezni, hiszen minden szempont nem fontos, és nem is vihető fel egy OO modellbe.
- Így alakul ki, hogy 3+1 lépés segítségével lehet VV objektumból, OO objektumot készíteni:
  - VV objektum felderítése
  - Egyszerűsítés, lényegkiemelés
  - OO típus létrehozása
  - OO objektum(ok) létrehozása



1. lépés - felderítés

- A folyamat első lépése a ValósVilág objektum meghatározása, felderítése.
- Meghatározzuk, hogy melyik az a valós világban létező objektum, melyet szeretnénk bizonyos szemléleti rendszer szerint beépíteni a saját objektumorientált programunkba.
- Pl.: Laptop VV objektumot szeretnénk beépíteni, mert Laptop kereskedés programját írjuk.

2. lépés - absztrakció

- Az objektumorientált programba való beépítés során nem vihetjük minden paraméterét a valósvilág objektumának, mivel egyrészről nem is kell, másrészről iszonyatos nagy tárigénye lenne (pl. több gigabájt, ha nem terrabájt lenne minden paraméterét letárolni egy laptopnak).
- Ezért kiszűrjük azon tulajdonságokat a VV objektumból amik (nem) szükségesek.
- Pl.: A Laptop(ok) esetében a program szempontjából érdekes lehet: gyártó neve, modell neve, cikkszáma, ára.

3. lépés - implementálás

- Miután megvan az egyszerűsített, lényeget tartalmazó változatunk, azt elkezdhetjük implementálni a megfelelő programnyelvbe, vagy program specifikációba, így a célkörnyezetben is alkalmazható a valósvilág objektum megfelelő szempontok szerinti változata.
- De ezzel még csak a lehetőség adott, maga az objektum ezen a szinten még nem jön létre.

3+1. lépés - példányosítás

- A folyamat utolsó lépése, amely azonban már programozási elem, és nem a VV → OO átírási folyamat része. Példányosítjuk a létrejött OO típust, melyből így valóban létrejön a valósvilágból kinyert objektumorientált objektum.
- Egy típus alapján több objektum is létrehozható, mindegyikük egyedi értékekkel, pl.:
  - `Laptop ujlaptop = new Laptop(”Samsung”, ”Series 9”, ”LTS001”, 300000);`
  - `Laptop ujlaptop2 = new Laptop(”HP”, ”Pavilion”, ”N3W03EA”, 350000);`

Tiszta kód IV.

- Az osztályok legyenek logikus felépítésűek a leszálló szabályt követve:
  - A felépítés sorrendje: publikus statikus állandók → privát statikus változók → privát példányváltozók → publikus alprogramok → privát alprogramok
  - Az osztály legyen minél kisebb, lehetőleg csak egy felelősségi körrel (feladattal) rendelkezzen.
  - Az osztály kevés példányváltozóval rendelkezzen, ahol minden tagmetódus használjon legalább egyet.
  - Ideális rendszerben az új szolgáltatásokat bővítéssel és nem a kód módosításával építjük be.
  - Az osztály és minden része legyen beszédes elnevezésű:
    - osztály, objektum → főnév; alprogram → ige, igenév
      - elérőfüggvény → get előtag // pl. getAddress()
      - módosítófüggvény → set // pl. setAddress()
      - lekérdezőfüggvény → is // pl. isChecked()

OOP Java-ban I.

- Java alapvetően OO nyelv, ennek megfelelően egy OO program fejlesztése lényegesen egyszerűbb benne, mint egy alapvetően nem OO nyelvben.
- Maga az IDE is segít az OO program kialakításában és gyors fejlesztésében, illetve esetleges objektumorientációs hibákat is azonnal jelez.

OOP Java-ban II.

- Osztályt létrehozni Java-ban a class kulcsszó segítségével tudunk.
- Pl.:
  - ```
    class PeldaOsztaly
    {
        //…
    }
    ```
    ```

Osztályok felépítése I.

- Az osztályok felépítése alapvetően a következőképpen néz ki:
  - Változó deklarációk (adattagok)
  - Konstruktor, [Destruktor]
  - Getter/setter
  - Metódusok
- Az osztály felépítésére nincs szabály, tehát akárhogy kinézhet, azonban a nem konzekvensen felépített osztályokat később nehéz újrafelhasználni.

Láthatósági szintek I.

- Minden elem, mely alprogramon kívül -de csomagon belül- jön létre, rendelkezik láthatósági szinttel.
- A láthatósági szint azt adja meg, hogy az adott elem melyik más programrészeknek elérhető (adatrejtés kritériumelem).
- A láthatóság megfelelő beállítása azért fontos, mert a csapatmunka és a továbbfejleszthetőség alapeleme az, hogy egyértelműen meghatározható legyen egy-egy elem elérhetősége.

Láthatósági szintek II.

- A láthatósági szintek emelkedő láthatósági sorrendben:
  - private
    - Csak az adott osztályon belül látható az adott elem.
  - nincs láthatósági szint:
    - A package-n belül mindenki láthatja az adott elemet.
  - protected
    - A package-n belül mindenki, másik package-ből pedig a leszármazottak láthatják az elemet.
  - public
    - Mindenkinek látható az adott elem.

Osztályok felépítése II. - változók

- Ahogy már megszokhattuk a változók az osztályokon belül is ugyanarra használhatóak, mint egy alprogramon belül.
- Az objektum szinten létrehozott változókat mezőknek, vagy field-nek hívjuk.
- A mező és a lokális változó közti legnagyobb különbség az élettartam és a láthatósági szint.
  - A láthatósági szintek állítása, az adatrejtés kritériumelem gyakorlati alkalmazása.

Osztályok felépítése III. - konstruktor

- A konstruktor egy speciális metódus, mely az osztály példányosításakor fut le.
- Feladata, hogy beállítsa az objektum kezdőértékeit.
- Használata nem kötelező, azonban konstruktor minden esetben szerepel egy osztályban, még akkor is ha nem írjuk meg.
  - → láthatatlan vagy default –paraméter nélküli- konstruktor

Osztályok felépítése IV. - konstruktor

- Felépítése:
  - ```
    <láthatósági szint> <Osztály neve>(<paraméterek>)
    {
        //programkód
    }
    ```
  - ```
    <láthatósági szint> <Osztály neve>()
    {
        //programkód
    }
    ```

Osztályok felépítése V. – getter/setter

- „Getter-Setter”-nek nevezzük azt a piciny metódus párost, mely biztosítja egy változó írhatóságát, olvashatóságát, vagy mindkettőt egyszerre.
- Használatával egy változó értékét csak olvashatóvá, vagy csak írhatóvá tehetjük a külvilág felé, vagy kontrollálhatjuk annak mikéntjét.

Osztályok felépítése VI. – getter/setter

- Formulája:
  - ```
    private <típus> <név>;

    <láthatósági szint> <típus> <getNév>()
    {
        return <név>;
    }

    <láthatósági szint> void <setNév>(<típus> value)
    {
        <név> = value;
    }
    ```
    ```

Osztályok felépítése VII. – metódusok

- Az alprogramok feladata ugyanaz, mint amit már megszokhattunk, azonban, ebben az esetben minden egyes alprogram speciális, hiszen azért helyezzük el az osztályban, mert egységbe zártuk, így akármilyen alprogram akárhová nem implementálható.
- Az alprogramok is rendelkeznek láthatósági szintekkel.